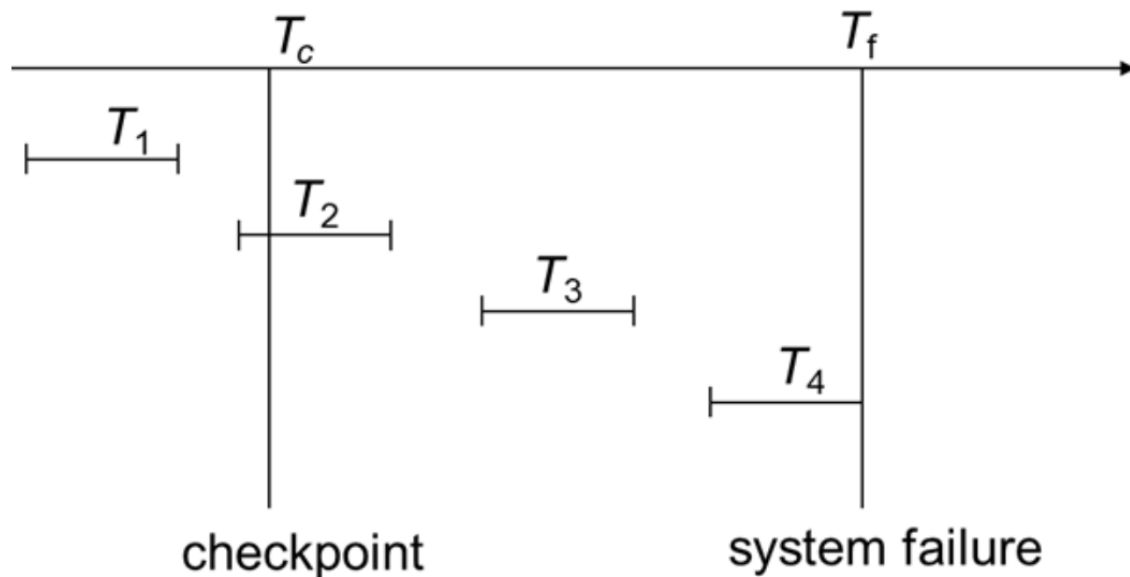


COMP90050 ADBS

Week 12

Q1 Crash Recovery

1. In the following figure the first vertical line T_c denotes the point where checkpointing was done and the second on the right, T_f , is where a system crash occurs. Please discuss what would change if the checkpointing was done right at the beginning of each transaction instead of the following case in the figure.



I.e. what is the benefit/cost of having more checkpoints

Benefit: Faster recovery

- After a system failure there would be less to do during recovery
- E.g. if a checkpoint was made at beginning of T_4 , then analysis phase does not need to consider T_3 which has already committed

Cost: Overhead during transaction processing

- Many output operations to disk for each checkpoint, and log entries regarding checkpointing

If immediate recovery is of utmost important, can increase frequency

If transaction processing should be done fast but recovery can take a long time, can reduce frequency

Q2 Distributed System

2. Discuss how timestamp strategy differs in concurrency control w.r.t its original form in distributed transactions.

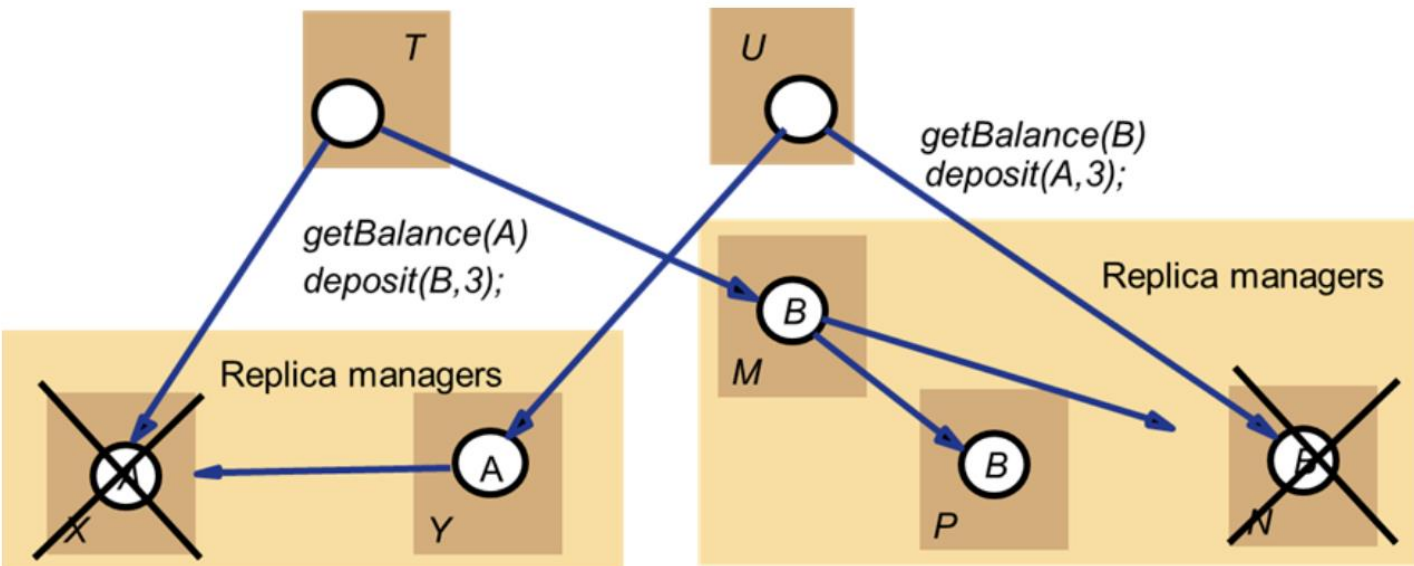
i.e. how is timestamp strategy done in a distributed database system?

- Each transaction is assigned a **globally unique timestamp** by the **coordinator**.
- The timestamp is passed with each object access.
- The **order** of the transactions' timestamps determines which transaction has the **priority to access objects at distributed servers**.
- Assures: if transaction X is before transaction Y based on the timestamp ordering, X will be before Y in their conflicting access to objects at all servers.

Q3 Replica & Available Copies strategy

3. Given the two following transactions T and U that run on replica managers X, Y, M, P, and N, we have seen in class a simple version of the Available Copies strategy that would not work. First review the problem that would occur if X and N were to crash during execution. Then state the solution that we have discussed in class. Last but not least, discuss what would happen under the final solution, if rather than X and N becoming unavailable we have the following scenario: If Y were to become unavailable during the execution and right after U accessed A at Y, but X and N do not fail, rest of the assumptions of this scenario is the same as we discussed in class.

Assume: X fails just after T performed getBalance; N fails just after U performed getBalance



If transactions simply access available servers:

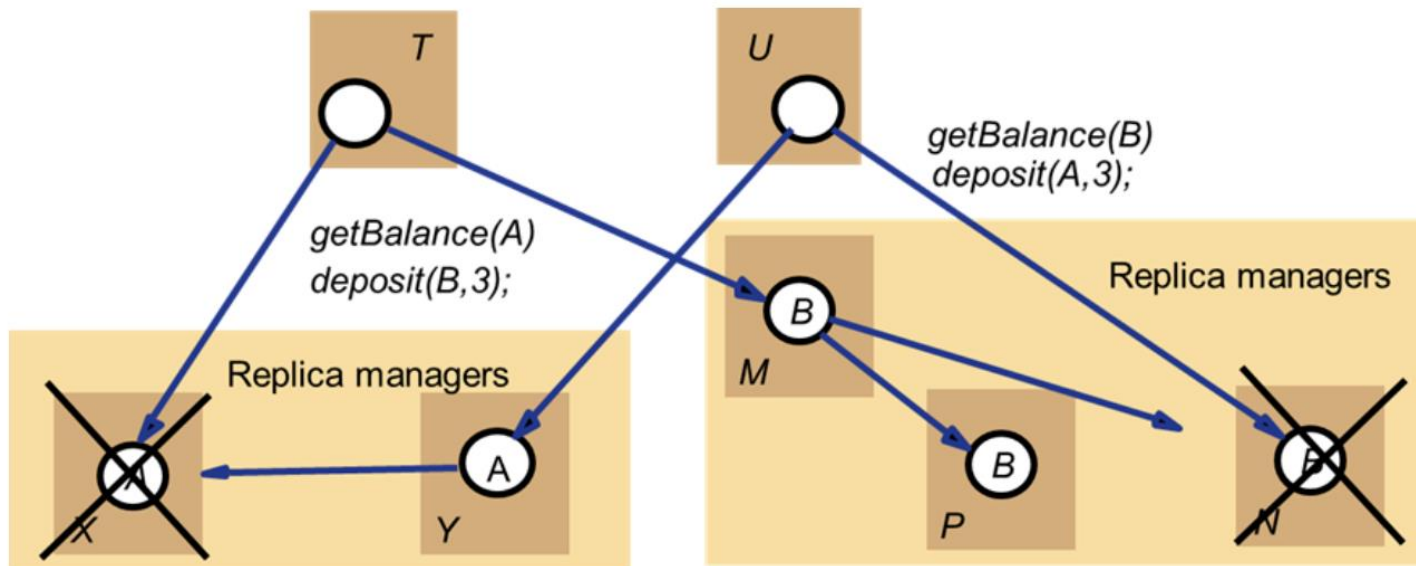
- U deposits A at Y
- T deposits B at M and P

Problem: isolation not preserved and T and U cannot be serialized

- U reads B before T's update & T reads A before U's update

Q3 Replica & Available Copies strategy

3. Given the two following transactions T and U that run on replica managers X, Y, M, P, and N, we have seen in class a simple version of the Available Copies strategy that would not work. First review the problem that would occur if X and N were to crash during execution. **Then state the solution that we have discussed in class.** Last but not least, discuss what would happen under the final solution, if rather than X and N becoming unavailable we have the following scenario: If Y were to become unavailable during the execution and right after U accessed A at Y, but X and N do not fail, rest of the assumptions of this scenario is the same as we discussed in class.



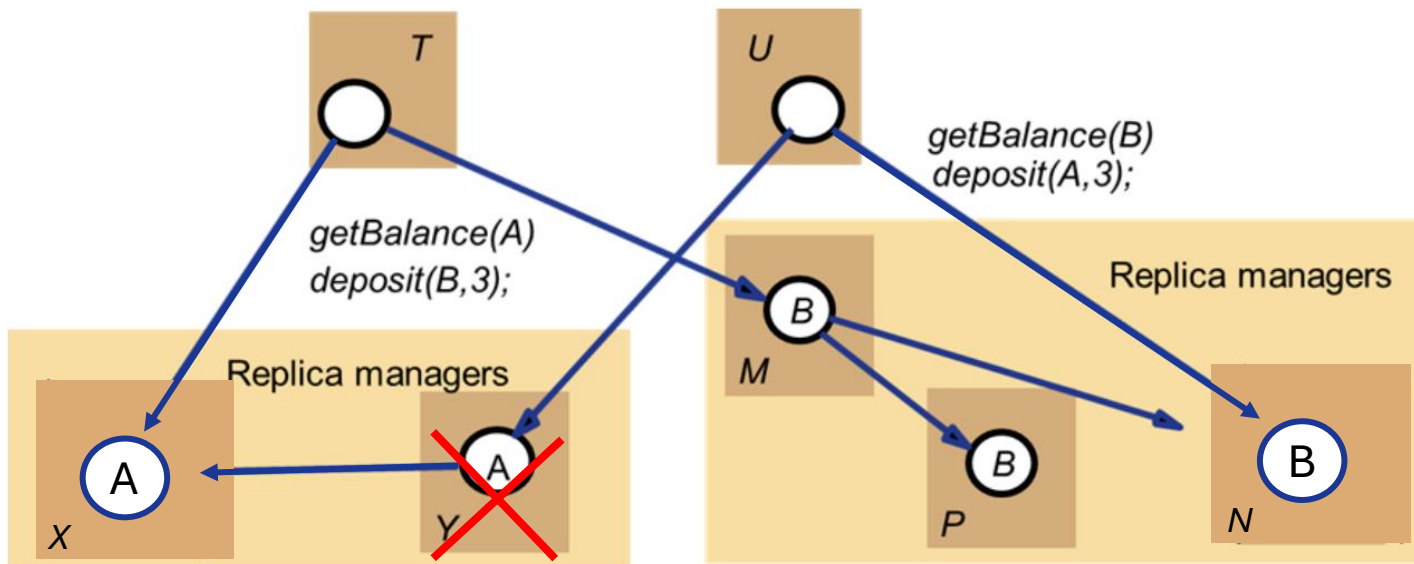
Available copies replication rule:

Before a transaction commits, it checks for failed and available servers it has contacted, the set should not change during execution.

E.g. T checks if X is still available, which failed before T's deposit. T would have to abort.

Q3 Replica & Available Copies strategy

3. Given the two following transactions T and U that run on replica managers X, Y, M, P, and N, we have seen in class a simple version of the Available Copies strategy that would not work. First review the problem that would occur if X and N were to crash during execution. Then state the solution that we have discussed in class. Last but not least, discuss what would happen under the final solution, if rather than X and N becoming unavailable we have the following scenario: If Y were to become unavailable during the execution and right after U accessed A at Y, but X and N do not fail, rest of the assumptions of this scenario is the same as we discussed in class.



Without the rule:

- U may lock Y and is delayed at X as A is locked by T
- On M,P,N:
 - If U gets lock first then T waits for access to B (while U waits for T), deadlock occurs and resolved by timer
 - If T gets lock first then U waits there until T finishes

No problem as T and U see each other

With the new rule: U will abort upon Y's failure

The implementation of the final strategy (and in fact many strategies in DBMS) are only approximations and in general pessimistic, to guarantee proper executions, but sometimes terminate transactions that would not really cause harm

Q4 Two-Phase Commit

4. Assume a two-phase commit involves a coordinator and three participants, P1, P2 and P3. What would happen in the following scenarios?
- Scenario 1: The coordinator crashed after receiving a 'yes' vote from all participants.
 - Scenario 2: P1 and P2 voted yes and P3 voted no.
 - Scenario 3: P2 crashed when it was about to send a vote message to the coordinator.

Phase 1: Voting

- Coordinator ask all participants to prepare to commit
- Each participant send vote message to coordinator & write prepare log to server

Phase 2: Commit

- If ALL participants voted commit, coordinator ask all participants to commit

If any participants abort / does not respond within timeout period, coordinator will abort

Q4 Two-Phase Commit

4. Assume a two-phase commit involves a coordinator and three participants, P1, P2 and P3. What would happen in the following scenarios?
- Scenario 1: The coordinator crashed after receiving a 'yes' vote from all participants.
 - Scenario 2: P1 and P2 voted yes and P3 voted no.
 - Scenario 3: P2 crashed when it was about to send a vote message to the coordinator.

Scenario 1: The main fallback is that the participants will send getDecision messages to the coordinator individually because they cannot receive further instruction from the coordinator. We assume that the coordinator eventually recovers from the crash to where it was.

Scenario 2: The coordinator will send doAbort messages to all participants. To achieve atomicity, all the participants should commit or none will commit.

Scenario 3: Since the coordinator cannot receive the vote from P2 within the timeout period, it will abort the transaction by sending doAbort messages to all participants.